

Tutorial – Week 07 5COSC019W – Object Oriented Programming

Arrays and Collections

03 November - 04 November

Array and ArrayList

Array :

- Arrays are simple data structures that store a set of values of the same type.
- Arrays are part of the Java language.
- Each value is accessed through an index
- Array index always starts at 0, not 1
- Array lengths cannot be changed once they are declared
- Arrays can be initialised when declared

ArrayList :

- It's a more flexible way to store data
- ArrayList can grow automatically as needed
- Has capacity that is increased when needed
- Has size() method that returns the actual number of elements in the ArrayList
- ArrayList class is not in the core Java language – It is in package java.util, which you must import:

```
import java.util.*; // At top of the program
```

Exercises

- 1) Consider the following method implementation. Try to figure out the solution without implementing the code. You can implement the code afterwards to verify if your solution is correct.

```
public static void loadArray(int[] list) {  
    for (int i = 1; i < list.length; i++)  
    {  
        list[i] = list[i] + list[i - 1];  
    }  
}
```

Indicate which values will be stored in the array after the method `loadArray` is executed if the following array of `int` is passed as a parameter to `loadArray`.

- a) {7}
- b) {3, 6}
- c) {4, 6, 8}
- d) {1, 2, 3, 4}
- e) {8, 4, 2, 0, 4}

- 2) Consider the following method implementation:

```
public static void listDemo(ArrayList<Integer> list) {  
    for (int i = 0; i < list.size(); i++)  
    {  
        int element = list.get(i);  
        list.remove(i);  
        list.add(0, element + 1);  
    }  
    System.out.println(list);  
}
```

Indicate which will be the output produced after the method `listDemo` is executed if the following `ArrayList` of `int` is passed as a parameter to `loadArray`.

- f) [5, 10, 15]
- g) [8, 2, 8, 7, 4]
- h) [-1, 3, 28, 17, 9, 30]

3) Create a new project in NetBeans and add a class `Book` to the Java project. The class should have:

- Four *instance variables*. They represent the title (`String`), the price (`double`), the year it was published (`int`) and the name of the author (`String`).
- One *constructor* will initialise the title, year, and name of the author.
- Write “set” and “get” methods for all instance variables.
- Write the `toString()` method for the book class that returns a string with the information about the book.

4) In the main method, write some Java code that saves in an `ArrayList` all the books that the user inserts from the keyboard. Implement a menu console from where the user can insert a book with the relative information (title, price, year, author’s name).

The user will be prompted something like:

Press 1 if you want to add a book

If the user insert 1, then the following will be asked:

Please insert the title of the book

<user answer>

Please insert the author of the book

<user answer>

Please insert the year of the book

<user answer>

Please insert the price of the book

<user answer>

Sorting an `ArrayList<Object>` with `Comparable`

5) Add a second option in your menu where the user can choose to print the list of books sorted by year:

Press 2 if you want to print the list of books sorted by year"

In order to fulfill this task, implement `Comparable` interface and then Override the `compareTo` method.

```
public class Book implements Comparable<Book>{  
    ...  
    ...  
    public int compareTo(Book b){  
        // compare here the book based on the year  
    }  
}
```

Now, in your main, you can call `Collections.sort` on `ArrayList` in order to sort your list of books.

- 6) You can try to modify the `compareTo` method to sort the books by title.

HashMap

`HashMap` maintains key and value pairs and is denoted as `HashMap<Key, Value>`. `HashMap` implements the `Map` interface. It is used to maintain key and value mapping.

It is not an ordered collection, which means it does not return the keys and values in the same order in which they have been inserted into the `HashMap`. You must need to import `java.util.HashMap` in order to use the `HashMap` class and methods.

- 7) Consider now that each book will be placed on a numbered shelf in a bookshop. Use Java map collection to map each book with the shelf. For each book, the user will enter the shelf number where it will be placed. Note that on one shelf, you can place many books.

Note that if you need to store primitive data types, you will need to use their corresponding wrapper class objects

Add a new option in the console menu:

Press 3 if you want to map all books into shelves

This is how you can use the `HashMap`:

```
/* This is how to declare HashMap */
HashMap<Book, Integer> hmap = new HashMap<Book, Integer>();

// insert the key for each book
for (int i=0; i<bookList.size(); i++){

    System.out.println ("Enter the number of the shelf where to place
    the book" + bookList.get(i).getTitle());

    int numShelf = s.nextInt();

    /*Adding elements to HashMap*/
    hmap.put(bookList.get(i), numShelf);
}
```



Key: the key is unique! In this case is the book entity

Value: the number of the shelf. In one shelf can be placed several books.

- 8) Add another option in the console menu that lets the user insert a shelf number from the keyboard, and then the list of books on that shelf will be printed.

Press 4 if you want to print the list of books on a specific shelf

Hint: you can iterate through all the values in a `HashMap` using an `Iterator`. An `Iterator` is an object that allows to cycle through a collection. Read carefully the following code on how to use an `Iterator`:

```
System.out.println("Insert the number of the shelf");
int selectedShelf = s.nextInt();
System.out.println("The book in shelf " + selectedShelf + " are:");
```

```
// Using entrySet() to iterate over the HashMap
for (Map.Entry<Book, Integer> entry : hmap.entrySet()) {
    if (entry.getValue().equals(selectedShelf))
        System.out.println(entry.getKey());
}
```

entrySet():

- Returns a Set containing all key-value pairs (Map.Entry<K, V>) in the HashMap.
- It allows you to **iterate** over the HashMap and access both keys and values at the same time.
- It provides an efficient way to **manipulate** or **process** all the entries in a HashMap.

9) Draw
a UML

Use Case Diagram for the system you just implemented.

Extra Exercises that you can do as an independent study

10) Implement a queue in Java using an Array or an ArrayList. Write the push and pop functionality.

You should implement a **Generic Class** so your queue will handle any type (e.g., String, int, instance of custom objects, etc.).

To implement a generic class, declare your Queue class like this and fill the body of the methods with the Java code:

```
public class Queue<T> {  
    private int maxSize;  
    private ArrayList<T> queue;  
  
    public Queue(int size) {...}  
    public void enqueue(T value) {...}  
    public T dequeue() {...}  
    public String toString() {...}  
}
```



T is the generic representation of the type!

In the method where you instantiate the Queue, include the type parameter T in the class definition. Here's how you can implement the main method:

```
public static void main(String[] args) {  
    Queue<String> newQueue = new Queue<>(5); // Specify type (e.g., String)  
  
    newQueue.enqueue("a");  
    newQueue.enqueue("b");  
    newQueue.enqueue("c");  
    newQueue.enqueue("d");  
    newQueue.enqueue("e");  
  
    System.out.println(newQueue.toString()); // Print the queue contents  
  
    System.out.println(newQueue.dequeue()); // Dequeue and print the  
    element  
  
    System.out.println(newQueue.dequeue()); // Dequeue and print another  
    element  
  
    System.out.println(newQueue.toString()); // Print the queue contents  
    again  
}
```

11) Singly Linked Lists are a type of data structure. In a singly linked list, each node stores its contents and a pointer or reference to the next node in the list. Watch the following video about the linked list:

<https://westminster.cloud.panopto.eu/Panopto/Pages/Viewer.aspx?id=1573494f-a6a7-47ac-98a7-af4100c16a5c>

According to what you learnt in the video (slides in Lecture Week 05 from slide 45 to slide 55), implement a generic Singly-Linked List.

- Specifically, implement a class Node that will have the contents of the node and a pointer (or reference) to the next node in the list. It does not store any pointer or reference to the previous

node. Write the get and set methods and a CompareTo method to compare if two Nodes are the same.

- Implement a SinglyLinked List. Remember that to store a single linked list, you only need to store a reference or pointer to the first node in that list. The last node has a pointer to nothing to indicate that it is the last node. Implement the following methods:

```
public void add(T newElement);  
public void addAfter(T newElement, T after);  
public void deleteFront();  
public void deleteAfter(T after);  
public void traverse();
```

- Implement a main class where you can instantiate the Linked list and try the operations you implemented.